

Kazihub Full-Stack Backend Requirements & Integration Specification

Version: 2.0.0 **Project:** Kazihub (Nigeria Escrow-Protected Artisan Marketplace) **Document Type:** Full-Stack Architecture, Database Schema, & Backend API Specification **Target Audience:** Backend Engineers, System Architects, Database Administrators, DevOps **Supersedes:** v1.0.0 — see [Appendix A: Changelog](#) for what changed and why.

1. Executive Summary & Architecture Overview

Kazihub is a hyper-localized Nigerian technical artisan and home-services marketplace connecting verified homeowners/clients with skilled trade artisans (Plumbers, Electricians, HVAC Specialists, Carpenters, Auto Mechanics, Welders, Painters, Appliance Technicians, Solar Installers, and Electronic Repairers).

The platform features:

- Escrow-Protected Transactions:** Customer funds are held securely until the client verifies job completion or until a 4-day automated dispute-expiry window lapses without objection.
- Dual-Pricing Mechanism:** Instant fixed-rate bookings for standard jobs alongside customizable request-for-quote (RFQ) workflows for complex or diagnostic work.
- Identity & Biometric KYC:** NIN, Voter's Card, Driver's License, or International Passport verification paired with a 3D facial liveness check to generate verified artisan trust badges.
- Rich Multi-Channel Communication:** In-app messaging, decibel-rendered voice notes, photo/video attachments, landmark location sharing, and real-time read receipts.
- AI Multimodal Diagnostic Engine:** Powered by Gemini to diagnose home maintenance issues from client descriptions or photos and recommend category, urgency, and estimated cost ranges.

Current System Status

The frontend is built with React 18, TypeScript, Tailwind CSS, TanStack Query, and Vite. The existing backend (`server.ts`) is currently an Express proxy hosting static assets and a single AI diagnosis route (`/api/diagnose`). All domain data (artisan profiles, bookings, chat histories, bookmarks, and notification logs) are hydrated from mock records and persisted client-side in `localStorage`.

Committed Architecture (decision made — do not re-litigate stack choice mid-build)

Layer	Choice	Rationale
Language	TypeScript, end-to-end	Matches the existing frontend; one type system, shareable DTOs.
API framework	Express + TypeScript (NestJS is an acceptable alternative <i>only</i> if the team already has NestJS experience — pick one before Sprint 1, do not run both)	Express keeps parity with the current <code>server.ts</code> proxy and lowers migration risk.
Database	PostgreSQL 15+	ACID transactions are mandatory for escrow — no NoSQL substitute.
DB host	Supabase (managed Postgres + built-in auth/storage helpers, but we run our own auth per §3, not Supabase Auth)	Fastest path to production for a small team; Cloud SQL is a fine self-hosted alternative if data-residency requirements change.
ORM / migrations	Prisma	Pick one ORM. Prisma's migration history doubles as schema documentation, which matters for an auditable financial system.
Object storage	Supabase Storage (S3-compatible) or AWS S3	Either works; must support pre-signed URLs and private buckets (see §8).
Cache / queues	Redis (BullMQ for background jobs)	BullMQ over Celery/Cloud Tasks, since the stack is Node-native.
Payments	Paystack (primary), Flutterwave as a documented fallback only if Paystack integration stalls	Avoid building two payment integrations in parallel.
Realtime	WebSocket via Socket.IO (or native <code>ws</code> behind the same Express server)	Needed for chat delivery, read receipts, and presence.
Search	PostgreSQL <code>pg_trgm</code> + GIN indexes for launch; Meilisearch if/when catalog size or query complexity outgrows Postgres full-text search	Don't reach for Elasticsearch on day one.

Architecture Objective

A relational database with strict ACID transactions for escrow state changes; a RESTful and WebSocket API layer enforcing JWT authentication, RBAC, and rate limiting; object storage for high-resolution images, video proofs, voice notes, and KYC documents; and integration with Paystack for escrow collection, split payments, and NUBAN-verified payouts.

2. System Roles, Permissions & Capability Model

Fix applied: v1.0.0 modeled role as a single column on `users`, which cannot represent a person who is *both* a client and an artisan — but the frontend's `kazihub_active_role` toggle (§8) requires exactly that. Identity and capability are now separated:

- Every account is a `users` row (identity).
- **Client** capability is implicit — any authenticated user can book a job.
- **Artisan** capability is granted the moment a user creates an `artisan_profiles` row, tracked in a `user_roles` join table so a single person can hold both capabilities and switch between them in the UI.
- **Admin** is a privileged flag (`users.is_admin`) that is **never** self-assignable through public registration — it is granted only via an internal admin-management tool or direct DB action by a super-admin, and requires mandatory 2FA (see §3).

client	Browse/search verified artisan profiles and gigs; submit fixed-price bookings and custom quote requests; accept quotes and fund escrow; exchange chat messages, audio voice notes, photos, and location coordinates; confirm job completion and release escrow; raise dispute tickets with evidence; submit ratings/reviews; manage own profile, avatar, language, and security settings.
artisan	Create/manage public trade profile, bio, services catalog, and rates; submit KYC documents and biometric liveness; receive booking/quote notifications; accept, decline, or quote incoming jobs; mark bookings "In Progress" and submit completion proof; toggle availability (Available)/(Busy)/(Offline); create/publish gigs; manage portfolio; withdraw earnings to a verified Nigerian bank account after escrow release.
admin	Audit/approve/reject KYC submissions; arbitrate escrow disputes; manually trigger partial/full refunds or release escrow (with mandatory reason + 2FA re-auth); suspend/reactivate/soft-delete fraudulent accounts; view audit logs, transaction volume, and dispute analytics. Every admin action that touches money or account status must write to audit_logs (§4.14) in the same DB transaction.

3. Authentication & Session Management Specification

Endpoints Required (all under **/api/v1/...** — see §5 for the versioning rationale)

POST	/api/v1/auth/register	-> Register user (client by default; artisan capability added via profile creation)
POST	/api/v1/auth/login	-> Authenticate with email/password; returns JWT + Refresh token
POST	/api/v1/auth/verify-email	-> Validate 6-digit OTP code sent via Email/SMS
POST	/api/v1/auth/resend-otp	-> Regenerate and dispatch OTP (rate-limited, see below)
POST	/api/v1/auth/forgot-password	-> Request password reset email/SMS link
POST	/api/v1/auth/reset-password	-> Submit token and set new password
GET	/api/v1/auth/me	-> Retrieve currently authenticated user profile + granted roles
PUT	/api/v1/auth/me	-> Update basic personal data (names, phone, state)
POST	/api/v1/auth/profile-picture	-> Multipart/form-data upload for user avatar
POST	/api/v1/auth/deactivate-me	-> Soft-delete + anonymize own account (see §12 for the single, authoritative deletion policy — replaces the "permanent purge" language from v1.0.0)
POST	/api/v1/auth/refresh	-> Exchange refresh token for new access token (rotation + reuse detection, see below)
POST	/api/v1/auth/revoke-sessions	-> Invalidate all active refresh tokens for the user

Authentication Rules & Security Requirements

1. **Passwords:** Hashed with **argon2id** (preferred) or **bcrypt** (work factor ≥ 12).
2. **JWT Tokens:**
 - Access Token:** 15-minute expiration. Signed with RS256. Payload contains **sub** (user ID), **roles** (array — e.g. **["client", "artisan"]**), **is_admin**, and **token_version**.
 - token_version:** an integer on **users**, incremented whenever an admin suspends/reactivates an account or the user revokes sessions. Middleware checks the token's **token_version** against the current DB value on every request, so a suspended account's still-valid access tokens are rejected immediately rather than waiting up to 15 minutes for natural expiry.
 - Refresh Token:** 30-day expiration, stored **hashed** (SHA-256) in **user_sessions**. Refresh uses **rotation with reuse detection**: each refresh issues a new token and invalidates the old one; if an already-used refresh token is presented again, the entire session family is revoked and the user is forced to re-login (this catches stolen-token replay).
 - Storage:** the current frontend keeps tokens in **localStorage**, which is vulnerable to XSS-based token theft. **Recommended improvement:** move the refresh token to an **httpOnly**, **Secure**, **SameSite=Strict** cookie, keeping only the short-lived access token in memory/localStorage. If the frontend timeline can't absorb this change now, flag it as fast-follow technical debt rather than closing the ticket.
3. **OTP Security:** 6-digit cryptographically random OTP, expires after 10 minutes, max 3 failed attempts before invalidation and forced regeneration. **resend-otp** and **login** are both rate-limited (see below).

4. Rate limiting (minimum bar for auth surface):

- `login`: 5 attempts / 15 min per IP+email combination, exponential backoff after that.
- `resend-otp`, `forgot-password`: 3 requests / hour per account.
- All other endpoints: global per-IP limiter (e.g. 100 req/min) plus per-user limiter once authenticated.

5. Webhook and payment endpoints are exempt from user-auth middleware but must independently verify signatures — see §10.

6. 401 / 403 Interception: When an access token expires or fails `token_version` validation, the client dispatches a refresh request; if refresh fails, all auth storage is wiped and the frontend dispatches `kazihub:unauthorized` to open the login modal.

7. Admin accounts require mandatory 2FA (TOTP) — `two_factor_enabled` is enforced server-side (not just offered) for any account with `is_admin = true`.

4. Database Entity-Relationship (ER) Schema Specification

A normalized relational schema in PostgreSQL. Every fix below is called out inline with a `-- FIX:` comment so the engineer can see exactly what changed and why relative to v1.0.0.

4.1. Table: `users`

sql

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  phone_number VARCHAR(30) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  -- FIX: role is no longer a single column; see user_roles below. is_admin is a  
  -- separate, non-self-assignable flag.  
  is_admin BOOLEAN NOT NULL DEFAULT FALSE,  
  -- FIX: NIN is sensitive PII (NDPR). Store encrypted, never plaintext.  
  nin_encrypted BYTEA,  
  -- FIX: no hardcoded state default — force the client to supply it, or leave  
  -- NULL until onboarding completes. A silent 'Oyo' default was masking missing data.  
  state VARCHAR(100),  
  lga_neighborhood VARCHAR(150),  
  avatar_url TEXT,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  is_email_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  is_phone_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  is_frozen BOOLEAN NOT NULL DEFAULT FALSE,  
  two_factor_enabled BOOLEAN NOT NULL DEFAULT FALSE,  
  two_factor_secret_encrypted BYTEA,  
  -- FIX: token_version supports immediate session invalidation (see §3).  
  token_version INT NOT NULL DEFAULT 0,  
  -- FIX: soft-delete + anonymization fields replace the "permanent purge" plan,  
  -- which was incompatible with ON DELETE RESTRICT on bookings (§4.6/§12).  
  deleted_at TIMESTAMPTZ,  
  anonymized_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE INDEX idx_users_active ON users(id) WHERE deleted_at IS NULL;
```

4.2. Table: `user_roles`

sql

```
-- FIX: new table. Lets one person hold both client and artisan capability,
-- matching the frontend's kazihub_active_role switcher.
CREATE TABLE user_roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL CHECK (role IN ('client', 'artisan')),
  granted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(user_id, role)
);
```

Every user implicitly has `client`; insert that row at registration. Insert `artisan` when `artisan_profiles` is created.

4.3. Table: `artisan_profiles`

```
sql

CREATE TABLE artisan_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  business_name VARCHAR(150),
  category VARCHAR(50) NOT NULL,
  tagline VARCHAR(255),
  bio TEXT,
  years_experience INT NOT NULL DEFAULT 1,
  hourly_rate NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  base_price NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  pricing_type VARCHAR(30) NOT NULL DEFAULT 'starting' CHECK (pricing_type IN ('fixed', 'starting', 'quote_
state VARCHAR(100) NOT NULL,
  neighborhood VARCHAR(150) NOT NULL,
  latitude DOUBLE PRECISION,
  longitude DOUBLE PRECISION,
  -- FIX: add a generated geography point + GIST index for real "nearby artisans"
  -- queries. Plain lat/lng columns with no index will not scale.
  geo_location GEOGRAPHY(POINT, 4326) GENERATED ALWAYS AS (
    ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)::geography
  ) STORED,
  is_available_now BOOLEAN NOT NULL DEFAULT TRUE,
  availability_status VARCHAR(20) NOT NULL DEFAULT 'Offline' CHECK (availability_status IN ('Available', 'B
is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  -- FIX: default rating/review count no longer implies a false 5-star reputation
  -- for brand-new artisans.
  rating_average NUMERIC(3, 2) NOT NULL DEFAULT 0.00 CHECK (rating_average BETWEEN 0 AND 5),
  review_count INT NOT NULL DEFAULT 0,
  completed_jobs_count INT NOT NULL DEFAULT 0,
  response_time VARCHAR(50),
  insurance_backed BOOLEAN NOT NULL DEFAULT FALSE,
  phone_visibility VARCHAR(30) NOT NULL DEFAULT 'after_escrow' CHECK (phone_visibility IN ('after_escrow', 'B
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_artisan_profiles_geo ON artisan_profiles USING GIST (geo_location);
CREATE INDEX idx_artisan_profiles_search ON artisan_profiles USING GIN (
  (business_name || ' ' || COALESCE(bio, '')) gin_trgm_ops
);

-- requires: CREATE EXTENSION IF NOT EXISTS pg_trgm; CREATE EXTENSION IF NOT EXISTS postgis;
```

4.4. Table: `artisan_services`

```
sql
```

```
-- Unchanged from v1.0.0 – no flaws found here.
CREATE TABLE artisan_services (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  artisan_profile_id UUID NOT NULL REFERENCES artisan_profiles(id) ON DELETE CASCADE,
  name VARCHAR(150) NOT NULL,
  category VARCHAR(50) NOT NULL,
  description TEXT,
  pricing_type VARCHAR(30) NOT NULL CHECK (pricing_type IN ('fixed', 'starting', 'quote_required')),
  price NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  duration_estimate VARCHAR(50) DEFAULT '1-2 hrs',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

4.5. Table: **artisan_portfolios**

sql

```
-- Unchanged from v1.0.0.
CREATE TABLE artisan_portfolios (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  artisan_profile_id UUID NOT NULL REFERENCES artisan_profiles(id) ON DELETE CASCADE,
  title VARCHAR(150) NOT NULL,
  category VARCHAR(50) NOT NULL,
  image_url TEXT NOT NULL,
  description TEXT,
  date_completed DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

4.6. Table: **gigs**

sql

```
CREATE TABLE gigs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  -- FIX: was users(id); now consistently keyed to artisan_profiles(id) like
  -- artisan_services and artisan_portfolios, so all artisan-owned catalog
  -- entities share one join pattern.
  artisan_profile_id UUID NOT NULL REFERENCES artisan_profiles(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  category VARCHAR(50) NOT NULL,
  price NUMERIC(12, 2) NOT NULL,
  delivery_time_days INT NOT NULL DEFAULT 1,
  tags TEXT[] DEFAULT '{}',
  images TEXT[] DEFAULT '{}',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  views_count INT NOT NULL DEFAULT 0,
  orders_count INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

4.7. Table: **bookings**

sql

```

CREATE TABLE bookings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reference_code VARCHAR(30) UNIQUE NOT NULL,
  client_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  artisan_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  service_id UUID REFERENCES artisan_services(id),
  gig_id UUID REFERENCES gigs(id),
  service_title VARCHAR(150) NOT NULL,
  category VARCHAR(50) NOT NULL,

  status VARCHAR(30) NOT NULL DEFAULT 'awaiting_quote' CHECK (
    status IN (
      'awaiting_quote', 'quote_submitted', 'pending_payment', 'pending_acceptance',
      'in_progress', 'completion_submitted', 'completed', 'disputed', 'cancelled'
    )
  ),

  pricing_type VARCHAR(30) NOT NULL,
  quoted_price NUMERIC(12, 2),
  escrow_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  platform_fee NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  -- FIX: track the payment gateway's own processing fee separately from
  -- platform commission – otherwise "artisan gets 90%" silently shrinks
  -- further once Paystack's fee is deducted, and nobody can see where the
  -- money went.
  gateway_fee NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  artisan_earnings NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
  -- FIX: commission rate is captured per booking, not just applied and
  -- forgotten – needed to explain historical payouts if the platform rate
  -- changes later.
  platform_commission_rate NUMERIC(5, 4) NOT NULL DEFAULT 0.10,
  escrow_status VARCHAR(30) NOT NULL DEFAULT 'unfunded' CHECK (
    escrow_status IN ('unfunded', 'held_in_escrow', 'released_to_artisan', 'refunded_to_client', 'partial'
  ),
  escrow_funded_at TIMESTAMPTZ,
  escrow_released_at TIMESTAMPTZ,
  -- FIX: optimistic-locking version column, used alongside row-level locking
  -- (see §6) so the auto-release cron and a client dispute can never both
  -- act on the same booking at once.
  lock_version INT NOT NULL DEFAULT 0,

  scheduled_date DATE NOT NULL,
  scheduled_time_slot VARCHAR(50) NOT NULL,
  location_address TEXT NOT NULL,
  location_landmark VARCHAR(255),
  latitude DOUBLE PRECISION,
  longitude DOUBLE PRECISION,
  client_name VARCHAR(150) NOT NULL,
  client_phone VARCHAR(30) NOT NULL,

  issue_description TEXT NOT NULL,
  problem_images TEXT[] DEFAULT '{}',
  landmark_images TEXT[] DEFAULT '{}',

  completion_submitted_at TIMESTAMPTZ,
  completion_description TEXT,
  completion_photos TEXT[] DEFAULT '{}',
  completion_video_url TEXT,
  auto_completion_deadline TIMESTAMPTZ,

```

```

cancellation_reason TEXT,
cancelled_by UUID REFERENCES users(id),
cancelled_at TIMESTAMPTZ,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

-- FIX: DB-level guard rail so application bugs can't leave the booking in
-- a self-contradictory state (e.g. completed with unfunded escrow).
CONSTRAINT chk_status_escrow_consistency CHECK (
    (status NOT IN ('completed') OR escrow_status IN ('released_to_artisan', 'refunded_to_client', 'parti
))
);

```

4.8. Table: **booking_status_history**

```

sql

-- FIX: new table. v1.0.0 promised a "timeline history" in GET /bookings/:id
-- but only ever stored the current status – there was nowhere to reconstruct
-- the timeline from.
CREATE TABLE booking_status_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    from_status VARCHAR(30),
    to_status VARCHAR(30) NOT NULL,
    changed_by UUID REFERENCES users(id),
    reason TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_booking_status_history_booking ON booking_status_history(booking_id, created_at);

```

4.9. Table: **disputes**

```

sql

-- Unchanged structurally; resolution now has a matching admin endpoint (§5.9).
CREATE TABLE disputes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ticket_id VARCHAR(30) UNIQUE NOT NULL,
    booking_id UUID UNIQUE NOT NULL REFERENCES bookings(id) ON DELETE RESTRICT,
    client_id UUID NOT NULL REFERENCES users(id),
    artisan_id UUID NOT NULL REFERENCES users(id),
    reason VARCHAR(100) NOT NULL,
    details TEXT NOT NULL,
    evidence_photos TEXT[] DEFAULT '{}',
    status VARCHAR(30) NOT NULL DEFAULT 'under_review' CHECK (
        status IN ('under_review', 'artisan_responding', 'arbitration', 'resolved_client_refund', 'resolved_a
    ),
    resolution_notes TEXT,
    resolved_by UUID REFERENCES users(id),
    resolved_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4.10. Table: **reviews**

```

sql

```

```
-- Unchanged from v1.0.0.
CREATE TABLE reviews (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id UUID UNIQUE NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  artisan_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
  comment TEXT,
  client_name VARCHAR(100) NOT NULL,
  is_verified_booking BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

4.11. Tables: `conversations` & `chat_messages`

sql

```
-- Unchanged structurally from v1.0.0. See §10 for the WebSocket auth fix.
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  artisan_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  booking_id UUID REFERENCES bookings(id) ON DELETE SET NULL,
  last_message_text TEXT,
  last_message_at TIMESTAMPTZ DEFAULT NOW(),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(client_id, artisan_id)
);

CREATE TABLE chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  recipient_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  text TEXT,
  media_url TEXT,
  media_type VARCHAR(30),
  audio_duration INT,
  audio_wave_data NUMERIC[],
  location_data JSONB,
  status VARCHAR(20) NOT NULL DEFAULT 'sent' CHECK (status IN ('sending', 'sent', 'delivered', 'read')),
  read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_chat_messages_conversation ON chat_messages(conversation_id, created_at);
```

4.12. Table: `verifications`

sql

```

CREATE TABLE verifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  -- FIX: dropped UNIQUE(user_id) so a rejected artisan can resubmit without
  -- destroying the rejection history. "Current" submission is the latest row.
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  document_type VARCHAR(30) NOT NULL CHECK (document_type IN ('nin', 'drivers_license', 'voters_card', 'pas
  -- FIX: encrypted at rest – this is a government ID number.
  document_number_encrypted BYTEA NOT NULL,
  document_image_url TEXT NOT NULL,
  liveness_selfie_url TEXT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'rejected')),
  rejection_reason TEXT,
  reviewed_by UUID REFERENCES users(id),
  reviewed_at TIMESTAMPTZ,
  -- FIX: explicit consent capture for biometric processing (NDPR requires
  -- documented consent for biometric data specifically).
  biometric_consent_given_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_verifications_current ON verifications(user_id, created_at DESC);

```

4.13. Table: **notifications**

```

sql

-- FIX: added delivery-channel tracking, since §11 dispatches SMS/WhatsApp in
-- addition to in-app notifications, and v1.0.0 had nowhere to record whether
-- an SMS actually sent.
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(40) NOT NULL,
  title VARCHAR(150) NOT NULL,
  message TEXT NOT NULL,
  booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,
  is_read BOOLEAN NOT NULL DEFAULT FALSE,
  channel VARCHAR(20) NOT NULL DEFAULT 'in_app' CHECK (channel IN ('in_app', 'sms', 'whatsapp', 'email')),
  delivery_status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (delivery_status IN ('pending', 'sent', 'fai
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4.14. Table: **audit_logs**

```

sql

```

```
-- FIX: new table. RBAC (§2) promises admins can "view audit logs" but
-- v1.0.0 never created one to view.
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id UUID NOT NULL REFERENCES users(id),
  action VARCHAR(60) NOT NULL, -- 'kyc_approved', 'dispute_resolved', 'escrow_force_released', 'user_suspen
  target_type VARCHAR(40) NOT NULL, -- 'booking', 'user', 'dispute', 'verification'
  target_id UUID NOT NULL,
  reason TEXT,
  metadata JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_audit_logs_target ON audit_logs(target_type, target_id);
```

4.15. Table: **saved_professionals**

sql

```
-- Unchanged from v1.0.0.
CREATE TABLE saved_professionals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  artisan_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(user_id, artisan_id)
);
```

4.16. Tables: **artisan_bank_accounts** & **escrow_transactions**

sql

```
CREATE TABLE artisan_bank_accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  bank_code VARCHAR(20) NOT NULL,
  bank_name VARCHAR(100) NOT NULL,
  account_number VARCHAR(20) NOT NULL,
  account_name VARCHAR(150) NOT NULL,
  paystack_recipient_code VARCHAR(100),
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE escrow_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE RESTRICT,
  transaction_reference VARCHAR(100) UNIQUE NOT NULL,
  gateway VARCHAR(30) NOT NULL DEFAULT 'paystack',
  amount NUMERIC(12, 2) NOT NULL,
  currency VARCHAR(10) NOT NULL DEFAULT 'NGN',
  type VARCHAR(30) NOT NULL CHECK (type IN ('deposit', 'release', 'refund', 'platform_fee', 'gateway_fee'))
  -- FIX: added 'reversed' to reflect that a completed Paystack transfer can
  -- still fail/reverse after the fact – v1.0.0 had no state for that.
  status VARCHAR(30) NOT NULL CHECK (status IN ('initiated', 'success', 'failed', 'reversed')),
  gateway_response JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

4.17. Table: `processed_webhook_events`

sql

```
-- FIX: new table, required for webhook idempotency (see §10 and §11). Without
-- this, a replayed Paystack webhook can double-fund or double-release escrow.
CREATE TABLE processed_webhook_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  gateway VARCHAR(30) NOT NULL,
  event_id VARCHAR(150) NOT NULL,
  event_type VARCHAR(60) NOT NULL,
  processed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(gateway, event_id)
);
```

5. Comprehensive REST & Realtime API Specifications

5.0. Cross-cutting conventions (new section — applies to every endpoint below)

- **Versioning:** every route is prefixed `/api/v1/...`. This lets us ship breaking changes as `/api/v2/...` later without a flag day.
- **Pagination:** list endpoints accept `limit` (default 20, max 100) and `offset` (or a `cursor` for high-traffic feeds). Responses use a standard envelope:

json

```
{ "data": [...], "meta": { "total": 132, "limit": 20, "offset": 0, "has_more": true } }
```

- **Errors:** standard shape across all endpoints:

json

```
{ "error": { "code": "BOOKING_NOT_FOUND", "message": "Human-readable message", "details": {} } }
```

- **Idempotency:** any `POST` that changes money or escrow state (`/bookings`, `/fund-escrow`, `/confirm-completion`, `/dispute`, admin refund/release) must accept an `Idempotency-Key` header; the server stores the key with the response and replays it on a duplicate request instead of re-executing the action.

5.1. Professionals & Search Discovery

- `GET /api/v1/profiles` — List artisans. Query params: `category`, `neighborhood`, `state`, `min_rating`, `min_experience`, `available_only`, `search` (uses the `pg_trgm` index from §4.3), `near_lat` `near_lng` `radius_km` (uses the PostGIS index), `limit`, `offset`.
- `GET /api/v1/profiles/:id` — Full public profile: portfolio, catalog services, reviews.
- `GET /api/v1/profiles/me` — Artisan's own private profile settings.
- `PUT /api/v1/profiles/me` — Update bio, rates, services, neighborhood, availability.
- `POST /api/v1/profiles/me/portfolio` / `DELETE /api/v1/profiles/me/portfolio/:id` — Manage portfolio items.

5.2. Gigs & Services

- `GET /api/v1/gigs`, `GET /api/v1/gigs/my`, `POST /api/v1/gigs`, `GET /api/v1/gigs/:id`, `PUT /api/v1/gigs/:id`, `DELETE /api/v1/gigs/:id` — unchanged from v1.0.0 aside from the `/v1` prefix.

5.3. Bookings & Escrow Lifecycle

- `POST /api/v1/bookings` — Client initiates booking/quote request. Requires `Idempotency-Key`.
- `GET /api/v1/bookings` — Scoped to authenticated user; filterable by `status`.
- `GET /api/v1/bookings/:id` — Full details, payment summary, and **timeline** — now sourced from `booking_status_history` (§4.8), so this actually returns a real audit trail.
- `POST /api/v1/bookings/:id/quote` — Artisan submits price estimate.

- `POST /api/v1/bookings/:id/accept-quote` — Client accepts; returns payment init payload.
- `POST /api/v1/bookings/:id/fund-escrow` — Requires `(Idempotency-Key)`. Confirms escrow payment after gateway callback (in addition to the webhook — see §10 for why both exist).
- `POST /api/v1/bookings/:id/accept` / `POST /api/v1/bookings/:id/decline` — Artisan response.
- `POST /api/v1/bookings/:id/submit-completion` — Starts the 4-day auto-release timer.
- `POST /api/v1/bookings/:id/confirm-completion` — Requires `(Idempotency-Key)`. Must acquire a row lock on the booking (see §6) before releasing escrow.
- `POST /api/v1/bookings/:id/dispute` — Requires `(Idempotency-Key)`. Must acquire the **same** row lock as the auto-release cron, so a dispute filed in the same instant the cron fires cannot lose the race silently.
- `POST /api/v1/bookings/:id/cancel`.

5.4. Chat & Real-Time Messaging

- `GET /api/v1/conversations`, `GET /api/v1/conversations/:id/messages`, `POST /api/v1/conversations/:id/messages`, `PATCH /api/v1/conversations/:id/read` — unchanged aside from `(/v1)` prefix.
- `WS /api/v1/ws/chat` — **Auth fix**: the client authenticates the WebSocket handshake by sending the access token in a `(Sec-WebSocket-Protocol)` subprotocol header (or a one-time short-lived WS ticket fetched via a prior authenticated REST call), **not** as a query string parameter — query strings end up in server access logs and browser history.

5.5. KYC & Artisan Verification

- `POST /api/v1/verification/submit` — Artisan submits document. Must include explicit biometric-consent acknowledgment, stored in `(biometric_consent_given_at)`.
- `GET /api/v1/verification/status`.
- `POST /api/v1/verification/:id/review` — Admin approve/reject. Writes to `(audit_logs)`.

5.6. Reviews & Ratings

- `POST /api/v1/reviews`, `GET /api/v1/reviews/pro/:proId` — unchanged.

5.7. Bookmarks & Saved Professionals

- `GET /api/v1/favorites`, `POST /api/v1/favorites/:proId`, `DELETE /api/v1/favorites/:proId` — unchanged.

5.8. Notifications

- `GET /api/v1/notifications`, `PATCH /api/v1/notifications/:id/read`, `PATCH /api/v1/notifications/read-all` — unchanged.

5.9. Admin (new section — closes the RBAC-vs-API gap from the audit)

- `GET /api/v1/admin/disputes` — List disputes, filterable by status.
- `POST /api/v1/admin/disputes/:id/resolve` — `{ resolution: 'client_refund' | 'artisan_paid' | 'split', split_ratio?, resolution_notes }`. Executes the corresponding escrow transfer/refund and writes `(audit_logs)`. Requires 2FA re-auth for the admin session.
- `POST /api/v1/admin/bookings/:id/force-refund` — Manual override outside the normal dispute flow, `{ amount, reason }`. Requires `(Idempotency-Key)` + 2FA re-auth + `(audit_logs)` entry.
- `POST /api/v1/admin/bookings/:id/force-release` — Same guard rails as above.
- `POST /api/v1/admin/users/:id/suspend` / `POST /api/v1/admin/users/:id/reactivate` — Sets `(is_active)` `(is_frozen)` and bumps `(token_version)` to immediately invalidate the user's live sessions. Writes `(audit_logs)`.
- `GET /api/v1/admin/audit-logs` — Filterable by `(actor_id)`, `(target_type)`, `(action)`, date range.
- `GET /api/v1/admin/analytics/overview` — Transaction volume, active disputes, KYC queue depth.

5.10. AI Problem Diagnosis

- `POST /api/v1/diagnose` — Analyzes symptom text or photo via Gemini. **Fix**: rate-limited per user (e.g. 10 requests/day on free tier) since each call has a real cost and no limit was specified in v1.0.0.

6. Escrow & Booking State Machine Lifecycle

The state diagram is unchanged from v1.0.0 (it was correct) — what's fixed is **how transitions are executed**, because that's where the money bugs live.

```
[Client Submits Job]
|
+---> Fixed Price Job -----> [Pending Acceptance] --(Artisan Accepts)--> [Pending Payment]
|
+---> Quote Required Job -----> [Awaiting Quote] --(Artisan Quotes)--> [Quote Submitted]
                                     --(Client Accepts)--> [Pending Payment]
                                     --(Client Funds Escrow)--> [In Progress]
                                     --(Artisan Finishes)--> [Completion Submitted]
                                     /(Client Confirms) (4-Day Timer Expires, No Dispute)\
                                     v                                     v
                                     [Job Completed] --(Escrow Released)--> [Review]

Prompt]
```

Concurrency requirements (fixes the critical race condition from the audit)

1. Every transition that touches `escrow_status` must run inside a single serializable-isolation DB transaction that:
 - Runs `(SELECT * FROM bookings WHERE id = $1 FOR UPDATE)` first, to take a row lock.
 - Re-checks `status` and `escrow_status` after acquiring the lock (not before) — this is what prevents the auto-release cron and a client's `(POST /dispute)` from both succeeding on the same booking.
 - Increments `lock_version` on write, so any concurrent request that read a stale version is rejected and must retry.
2. The auto-release cron (§7) must skip — not error on — any booking it cannot lock immediately (i.e. use `(FOR UPDATE SKIP LOCKED)` when scanning the batch), and pick it up on the next run rather than blocking the whole batch on one contested row.
3. Every status change writes a `booking_status_history` row in the same transaction as the status update — no exceptions, including for system-initiated (cron) transitions.

Dispute Path

From `[Completion Submitted]` or `[In Progress]`, the client calls `(POST /api/v1/bookings/:id/dispute)`:

- Acquires the row lock described above.
- Transitions to `[Disputed]`; the 4-day timer is implicitly frozen because the auto-release cron's query filter excludes `status = 'disputed'`.
- Creates a `disputes` row with evidence.
- Notifies both parties in-app and by email.
- Admin resolves via `(POST /api/v1/admin/disputes/:id/resolve)` (§5.9), which is now a real endpoint instead of an implied one.

7. Background Jobs & Scheduled Tasks

Queue/worker: **BullMQ on Redis** (see §1 for why this replaces the Celery/Cloud Tasks "or").

1. **4-Day Escrow Auto-Release Job** — runs hourly. Queries bookings where `status = 'completion_submitted' AND NOW() >= auto_completion_deadline`, using `(FOR UPDATE SKIP LOCKED)` (§6). For each: marks `completed`, releases escrow, writes `booking_status_history`, notifies the client.
2. **OTP & Token Clean-up** — purges expired OTPs and refresh tokens older than 60 days.
3. **SMS / WhatsApp Milestone Dispatches** — via Termii or Twilio to `(+234...)` numbers; writes delivery outcome to `notifications.delivery_status` (§4.13).
4. **Webhook Retry Processor** — (new) if a Paystack webhook handler fails after signature verification (e.g. DB hiccup), the event is re-queued with exponential backoff rather than silently dropped, and checked against `processed_webhook_events` on each retry to stay idempotent.
5. **Reliability requirement** (new): every scheduled job must alert on failure (Slack/PagerDuty webhook) — a silently-failing auto-release cron means client money sits in limbo with no one aware.

8. Client-Side Persistence Migration Audit

Unchanged in substance from v1.0.0 — the mapping was sound. Updated only to reflect the `/v1` prefix and the role-model fix:

Frontend <code>localStorage</code> Key	Current Frontend Storage	Target Database Table / Field	Migration Action Required
<code>kazihub_ng_professionals_v11</code>	List of artisans in <code>App.tsx</code>	<code>artisan_profiles</code> + <code>users</code> + <code>artisan_services</code>	Seed from <code>mockData.ts</code> ; load via <code>GET /api/v1/profiles</code> .
<code>kazihub_ng_bookings_v12</code>	Booking records array	<code>bookings</code> table	Replace with <code>useQuery(['bookings'])</code> → <code>GET /api/v1/bookings</code> .
<code>kazihub_ng_messages_v11</code>	Chat messages array	<code>conversations</code> + <code>chat_messages</code>	Replace with query + WebSocket events.
<code>kazihub_saved_pros</code>	Array of pro IDs	<code>saved_professionals</code>	<code>GET /api/v1/favorites</code> ; toggle via <code>POST/DELETE /api/v1/favorites/:proId</code> .
<code>kazihub_recently_viewed</code>	Array of viewed artisan IDs	Redis session cache	Keep in sessionStorage, or persist to a short-TTL Redis key per user.
<code>kazihub_recent_searches</code>	Array of search strings	<code>user_search_history</code> table	Sync to profile settings, or retain client-side only.
<code>kazihub_kyc_completed_\${id}</code>	Boolean string	<code>verifications.status = 'approved'</code>	<code>GET /api/v1/verification/status</code> .
<code>kazihub_artisan_status_\${id}</code>	Availability enum	<code>artisan_profiles.availability_status</code>	<code>PUT /api/v1/profiles/me</code> .
<code>kazihub_customer_avatar</code>	Data URL	<code>users.avatar_url</code>	<code>POST /api/v1/auth/profile-picture</code> .
<code>kazihub_customer_notifications</code> / <code>kazihub_pro_notifications</code>	Arrays in <code>App.tsx</code>	<code>notifications</code> table	<code>GET /api/v1/notifications</code> + WebSocket/SSE push.
<code>kazihub_theme</code>	Light/dark	Client preference	Keep local for instant load; optionally sync to <code>users</code> .
<code>kazihub_active_role</code>	<code>'customer'</code> <code>'artisan'</code>	<code>user_roles</code> (fix applied)	Derive available roles from <code>GET /api/v1/auth/me</code> ; the switcher toggles which UI is shown, it no longer needs to mutate a single <code>role</code> column.

9. Media & File Storage Pipeline

Same five touchpoints as v1.0.0 (avatars, problem photos, landmark photos, completion proof, KYC docs, voice notes), with these fixes:

- `POST /api/v1/uploads/presigned-url` now validates `content_type` against an allow-list (`image/jpeg`, `image/png`, `image/webp`, `video/mp4`, `audio/webm`, `audio/wav`) and a declared `max_size_bytes` **server-side** before issuing the

URL — v1.0.0 issued pre-signed URLs with no size/type gate, allowing arbitrary uploads.

- Image optimization (Sharp or a Cloud Function) resizes to max 1920×1080 and converts to WebP.
 - KYC files live in a **private** bucket, accessible only via 15-minute signed URLs for admin review.
 - **New:** all uploaded images (not just KYC) pass through basic content moderation (NSFW/illegal-content classifier) before being marked visible in chat or portfolios — chat photo/video attachments were previously unmoderated.
-

10. Realtime Chat & Audio Notes Engine

Unchanged in data model from v1.0.0. Fixes:

1. **WebSocket authentication** — see §5.4: token passed via subprotocol header or a short-lived WS ticket, not a query string.
 2. **Voice notes** — stored as before (`audio_wave_data: number[]`), duration in seconds). No change needed here; storage cost is small enough not to warrant downsampling at this stage.
 3. **Read receipts** — (`sending → sent → delivered → read`), (`mark_read`) emitted over WebSocket on chat open.
 4. **Live geolocation** — (`location_data JSONB`) with (`{ lat, lng, addressName, landmark }`), unchanged.
-

11. Financial Transactions & Escrow Integration (Nigeria)

1. **Payment Gateway:** Paystack primary (§1).
 2. **Deposit Flow:** unchanged — (`POST https://api.paystack.co/transaction/initialize`) on quote/booking confirmation; Paystack popup/redirect; (`charge.success`) webhook updates escrow state.
 3. **Webhook handling — fixed:**
 - (`POST /api/v1/webhooks/paystack`) verifies the (`x-paystack-signature`) header against the request body using the webhook secret **before** touching the DB.
 - The event's Paystack event ID is checked against (`processed_webhook_events`) (§4.17); if already present, the handler returns (`200 OK`) immediately without re-processing (idempotency — closes the double-fund/double-release bug from the audit).
 - Handles (`charge.success`), (`transfer.success`), (`transfer.failed`), and (`transfer.reversed`) — v1.0.0 only handled the deposit side; a failed/reversed payout now flips (`escrow_transactions.status`) to (`'failed'`) (`'reversed'`) and re-queues the payout for retry instead of silently marking the booking (`completed`) with money that never arrived.
 4. **Disbursement Flow:** on completion, trigger Paystack Transfer API to (`artisan_bank_accounts.paystack_recipient_code`). Deduct **both** (`platform_commission_rate`) (captured per-booking in (`bookings`), §4.7) **and** the actual Paystack transfer fee (`gateway_fee`) — v1.0.0 only accounted for platform commission and implicitly assumed the artisan got the full 90%, ignoring Paystack's own cut.
 5. **Bank Account Verification (NUBAN):** (`POST /api/v1/payments/verify-bank-account`) → Paystack NUBAN resolve, unchanged.
 6. **Ledger integrity:** (`escrow_transactions`) remains the append-only event log; reconciliation reports (for admin analytics, §5.9) should sum (`escrow_transactions`) by (`booking_id`) and (`type`) rather than trusting (`bookings.escrow_status`) alone, so a discrepancy between the two is itself a detectable signal of a bug.
-

12. Security, Privacy & Compliance (NDPR)

Single, authoritative account-deletion policy (this replaces the contradictory language in v1.0.0 §3 vs §12):

- (`POST /api/v1/auth/deactivate-me`) sets (`deleted_at = NOW()`) and immediately revokes all sessions (bump (`token_version`)).
- A background job, after a grace period the business defines (recommend confirming an exact retention window with legal/compliance counsel — this document doesn't set one unilaterally), runs anonymization: overwrites (`first_name`), (`last_name`), (`email`), (`phone_number`), (`nin_encrypted`), and avatar with anonymized placeholders, sets (`anonymized_at`), and deletes KYC document/selfie files from object storage.
- **Bookings, reviews, and dispute records are never hard-deleted** — (`ON DELETE RESTRICT`) stays in place. This is what "right to be forgotten" actually means here: the PII is scrubbed, the transactional/audit trail (required for financial recordkeeping and fraud investigation) remains.

- Data export: `(GET /api/v1/users/me/export-data)` returns the full JSON archive (profile, bookings, disputes, reviews) — unchanged from v1.0.0, this part was fine.

Additional fixes:

- `(nin_encrypted)`, `(document_number_encrypted)`, and `(two_factor_secret_encrypted)` are encrypted at the application layer (e.g. AES-256-GCM with a key from a secrets manager, not `(pgcrypto)` with an in-DB key) before insert, so a DB dump alone doesn't expose them.
 - Biometric consent is explicitly captured (`(verifications.biometric_consent_given_at)`) at submission time, satisfying NDPR's heightened requirement for special-category data.
 - Phone number visibility rules (`(after_escrow)` / `(verified_only)` / `(hidden)`) — unchanged from v1.0.0, masking logic applied in `(GET /api/v1/profiles/:id)`.
 - **Recommendation, not yet a hard requirement:** confirm KYC document retention period with legal counsel and Nigerian data-protection guidance before finalizing the anonymization job's grace period — this document intentionally does not assert a specific number of days, since that's a compliance decision rather than an engineering one.
-

13. Frontend-to-Backend Integration Checklist

- ☐ **Step 1: Auth Integration** — Replace demo mock logins with `(authApi.login())`/`(register())`; wire `(apiClient)` interceptors for rotation-with-reuse-detection refresh; handle `(token_version)` mismatch as a forced logout.
 - ☐ **Step 2: Role Model Migration** — Replace any assumption of a single `(user.role)` with `(GET /api/v1/auth/me)` returning a `(roles: string[])` array; `(kazihub_active_role)` becomes a pure UI-state toggle, not a server mutation.
 - ☐ **Step 3: Profile & Catalog Migration** — Seed DB with the 10 verified artisans from `(mockData.ts)`; query via `(useListProfiles())` against `(GET /api/v1/profiles)`.
 - ☐ **Step 4: Booking & Escrow Flow** — Wire `(BookingModal.tsx)` to `(bookingApi.createBooking())` with an `(Idempotency-Key)`; implement the Paystack popup on quote acceptance; handle the new `(gateway_fee)` field in any earnings breakdown UI.
 - ☐ **Step 5: Messaging & WebSocket Sync** — Replace local `(messages)` state with WebSocket events + `(GET /api/v1/conversations/:id/messages)`; update the WS connection to authenticate via subprotocol/ticket, not a query string.
 - ☐ **Step 6: KYC Verification Modal** — Replace the `(setTimeout)` simulation with `(verificationApi.submitVerification())`, including the new biometric-consent checkbox required before submission.
 - ☐ **Step 7: Settings, Reviews & Account Lifecycle** — Connect dispute modal to `(POST /api/v1/bookings/:id/dispute)`; replace any "delete my account" button copy/flow to reflect deactivation + anonymization (not instant permanent deletion) per §12.
 - ☐ **Step 8: Admin Console** (*new — v1.0.0 had no admin UI plan*) — Build the admin screens needed to call `(POST /api/v1/admin/disputes/:id/resolve)`, `(force-refund)`, `(force-release)`, `(suspend)`/`(reactivate)`, and view `(GET /api/v1/admin/audit-logs)`.
 - ☐ **Step 9: AI Diagnosis Endpoint** — Verify `(/api/v1/diagnose)` passes symptom text/image to Gemini and respects the new per-user rate limit.
-

Appendix A: Changelog from v1.0.0

#	Issue	Fix
1	Single <code>role</code> column couldn't support the dual client/artisan UX	Split into <code>user_roles</code> join table + separate <code>is_admin</code> flag
2	Contradictory account-deletion policy (hard purge vs. anonymize)	Single policy in §12: soft-delete + scheduled anonymization, transactional records retained
3	No concurrency control on escrow transitions (cron vs. dispute race)	Row locking (<code>FOR UPDATE</code>) / (<code>SKIP LOCKED</code>) + <code>lock_version</code> + serializable transactions, §6
4	No webhook idempotency	<code>processed_webhook_events</code> table + signature verification, §10/§11
5	RBAC promised admin capabilities with no matching endpoints	Added full §5.9 admin API + <code>audit_logs</code> table
6	No ledger visibility into payout failures	<code>escrow_transactions.status</code> now includes <code>reversed</code> ; <code>transfer.failed</code> / <code>transfer.reversed</code> webhooks handled
7	PII/biometric data stored in plaintext	<code>nin_encrypted</code> , <code>document_number_encrypted</code> , <code>two_factor_secret_encrypted</code>
8	New artisans defaulted to a fake 5.0 rating	Default <code>0.00</code> , <code>CHECK</code> constraint added
9	No geospatial or full-text search indexing	PostGIS <code>geo_location</code> + GIST index, <code>pg_trgm</code> GIN index
10	Inconsistent FK targets (<code>gigs.artisan_id</code> vs. <code>users.id</code>)	Standardized to <code>artisan_profiles.id</code> for all artisan-owned catalog entities
11	No booking timeline storage despite API promising one	<code>booking_status_history</code> table
12	Unbounded pagination	<code>limit</code> / <code>offset</code> with max cap + standard response envelope, §5.0
13	WebSocket token passed via query string	Moved to subprotocol header / short-lived ticket
14	<code>verifications.user_id</code> UNIQUE blocked resubmission after rejection	Constraint relaxed; latest row by <code>created_at</code> is authoritative
15	No stack commitment (multiple ORMs/DBs/frameworks listed as "or")	Committed stack in §1
16	Platform commission hardcoded with no per-booking record	<code>platform_commission_rate</code> and <code>gateway_fee</code> captured on the <code>bookings</code> row itself
17	Presigned upload endpoint had no server-side validation	Content-type allow-list + size limit enforced before issuing the URL

Specification revised following a full audit of v1.0.0 against production-readiness, security, and NDPR compliance best practices.